

Multyr Keeper Workers (W1–W6)

Test report · Arbitrum One deployment of 2026-09-25 · prepared 2026-10-07

Interim report. All tests were run against local Anvil forks of Arbitrum One. No mainnet transactions were sent. The historical keeper cycle (rows 6–56 of the adapter-cycle PDF driven by keepers) has not finished yet; section 5 lists what is still open.

160/160

Solidity tests (forge)

22/22

Keeper unit tests (+1 todo reproducing a bug)

6/6

Workers exercised with real transactions on a fork

3

Issues to fix before go-live

1. The workers

The keeper makes no allocation decisions. It reads on-chain state, simulates each call from the keeper address, and only sends a transaction when the contract would accept it. All eligibility rules stay on-chain.

ID	Contract	Call	Permission	Trigger
W1	VaultUpkeep 0x0196...2FFE	checkUpkeep → performUpkeep	none	poll 120 s, up to 10 follow-up actions per tick
W2	StrategyUpkeep 0xA01A...3ee6	checkUpkeep → performUpkeep	none	poll 300 s; on-chain pokeInterval (86 400 s) and cooldowns
W3	ClaimSettlementUpkeep 0xD8CE...a14E	checkUpkeep → performUpkeep	none	poll 120 s, up to 25 batches per tick
W4	FeeCollector 0x7976...4B9a	harvestQueued(token), distribute(token)	none	harvest when claims are pending; distribute Mon 10:00–16:00 UTC
W5	Strategy vault 0xCC4A...AEf1	pokeLiquidityBatch(start,end)	KEEPER_ROLE per keeper address	batches whose liquidity cache is within 300 s of expiry
W6	BufferManager 0xEC5a...57e1	refreshWarmNav()	none	cache within 120 s of navRefreshInterval (600 s), or invalid

Primary / secondary. Both bots run the same image with different keys and RPC providers. The primary acts as soon as a condition is due; the secondary acts only after the condition has stayed due for that worker's grace window.

2. What was tested

Test	Where	Result
Keeper unit tests: helpers, gate, KMS signing, and every worker's tick() against a scripted chain	local	22 pass, 0 fail, 1 todo
Typecheck	local	clean
Solidity suite (8 suites)	local	160 / 160

Preflight: chain ID, code at all 8 addresses, wiring, roles, config	live state, block 512 386 690	wiring correct
Dry run of every worker	live state	W1, W2, W6 due; W5 needs role
Real transactions, time warps, primary/secondary failover	fork at latest block	see section 3
Queue scenario: fees, epoch close/fund, claim payout, fee harvest	fork at block 509 739 747	see section 4

3. Results by worker (fork at latest block)

Worker	Result	Evidence
W1	Fails with bot gas	CRYSTALLIZE inner call ran out of gas three times in one tick, which put VaultUpkeep into its 30-minute backoff. With enough gas, CRYSTALLIZE and REBALANCE both succeed.
W2	Partial with bot gas	POKE_APY mined but emitted ExternalTVLPokeFailed . With enough gas: POKE_APY then HARVEST, no failure events.
W3	Pass (section 4)	Nothing due at the latest block; claim payout verified on the historical fork.
W4	Pass	Distributed 59 816 core shares 70 / 29 / 1. With 1 000 USDC funded: nothing outside the window; 700 / 290 / 10 USDC inside it; nothing on re-run.
W5	Pass	Refreshed all 7 adapter caches in two batches. Missing KEEPER_ROLE raises an alert and stops.
W6	Pass	Refreshes when due; idempotent on re-run.
Failover	Pass	Secondary deferred with 156 s and 76 s of grace left while the primary acted; with the primary absent it took over after the grace window. No duplicate transactions.

Gas needed versus the bot's estimate

Operation	Bot estimate	Actually needed	Shortfall
W1 CRYSTALLIZE	975 649	1 104 746	13 %
W2 POKE_APY	1 577 639	2 072 735	31 %

4. Queue scenario (historical fork, block 509 739 747)

This scenario starts from the state just before the adapter-cycle PDF, replays its first five transactions, then drives a full deposit → fee → withdrawal → payout cycle with the production worker classes. Fork-only setup: AUTO_HARVEST enabled on the FeeCollector, 1 bps instant-withdrawal cap, zero deposit lock, a fresh keeper wallet with KEEPER_ROLE, a 10 gwei local fee cap, and the USDC price feed's timestamp advanced for the 7-day jump (price, decimals and round IDs preserved). Calls in this pass used a fixed 6 000 000 gas budget, to separate the gas-estimate bug from contract behaviour.

#	Worker	Action	Gas used	Outcome
1	W6	<code>refreshWarmNav</code>	99 540	valid
2	W5	<code>pokeLiquidityBatch(0,4)</code>	332 950	no adapter failures
3	W5	<code>pokeLiquidityBatch(4,7)</code>	236 574	no adapter failures
4	W4	<code>distribute(coreShares)</code>	1 148 117	queued 9 990 fee shares for harvest
–	user	<code>requestEpochWithdrawal</code>	736 097	queued withdrawal
<i>Time advanced one epoch (7 days)</i>				
5	W6	<code>refreshWarmNav</code>	99 540	valid
6	W1	<code>EPOCH_CLOSE</code>	541 612	success
7	W1	<code>EPOCH_FUND</code>	705 044	success
8	W1	<code>CRYSTALLIZE</code>	837 726	success; 59 fee shares minted
9	W3	<code>SETTLE_CLAIMS</code>	645 443	2 claims paid, 3.989998 USDC (user 3.980024, FeeCollector 0.009974)
10	W4	<code>harvestQueued(coreShares)</code>	112 103	9 990 shares → 9 974 USDC units, distributed 6 981 / 2 894 / 99
11	W4	<code>distribute(coreShares)</code>	≈1.1 M	queued a new claim for the freshly minted fee shares

Why the earlier run reported a W4 failure

The test asserted that no fee claim was pending after step 10. The 7-day jump landed inside the Monday distribution window, so the same tick also ran step 11: it distributed the fee shares that CRYSTALLIZE had just minted and, under AUTO_HARVEST, queued a new claim for them. That is correct behaviour. The assertion has been changed to check that the original claim was settled in full, the USDC was distributed, nothing was left in the FeeCollector, and any claim still pending was queued in that same tick. **W4 itself needed no change.**

5. Issues and open items

Fix before go-live

- Gas estimate too low for try/catch operations (keeper).** The estimate stops at the lowest gas where the outer call succeeds, which starves the inner call. W2 is the worse case: a failed POKE_APY still sets `lastPokeTs`, so it is not retried for 24 h. Fix: add a gas buffer (for example estimate × 1.5) or a per-worker gas limit.
- Follow-up loop re-sends a failed operation (keeper).** After `UpkeepPerformed(success=false)`, `checkUpkeep` stays true and W1 repeats the same failing op until `VaultUpkeep` backs off. Fix: stop the loop on an inner failure. A unit test (todo) reproduces this.

3. **VaultUpkeep grieving (core contract).** Any address can send three `performUpkeep` calls with junk data and freeze VaultUpkeep for 1 800 s, repeatedly. Confirmed on the fork. Fix: re-validate `performData` against `checkUpkeep` and do not count mismatches as failures.

Review

- Fee cap compares the full fee estimate including the priority tip. Arbitrum suggests a zero tip today, but a provider that suggests a non-zero tip would make the bot skip every transaction.
- `AUTO_HARVEST` is not configured for the core share token on mainnet, so W4's `harvestQueued` cannot run until governance sets it.
- `liquidityStalenessSeconds` is 0 on-chain; W5 falls back to 3 600 s.
- Warm NAV reports `nav = 0` with `valid = true` on mainnet; confirm this is expected.
- Four of seven adapters are not direct APY poke targets for W2: Aave (covered by the rate push), Comet, Euler and Venus. Confirm the last three compute APY live.

Still to run

- Re-run the historical keeper cycle with the corrected W4 check, through the PDF adapter cycle (rows 6–56 driven by W1/W2).
- Re-test W1 and W2 with the bot's own gas estimate after fixes 1 and 2.
- Second RPC provider for the AWS primary (the QuickNode URL supplied returned 401).

Local fork tests only. Contract addresses are the Arbitrum One deployment of 2026-09-25. Gas figures are from fork receipts and can differ slightly on mainnet.